Checkpoint C — Quality and Residual Risk

QuantRule

2026-11-11

Contents

1 Checkpoint C — Quality and Residual Risk **1**
1.1 Claim . . . 1
1.2 Risk map . . . 1
1.3 One behaviour . . . 1
1.4 Review and AI use . . . 2
1.5 Decision . . . 2

1 Checkpoint C — Quality and Residual Risk

1.1 Claim

The Tauri Platform's highest-risk behaviours have executable evidence and a recorded residual risk.

1.2 Risk map

Risk	Control	Evidence
Reproducibility drift	Golden and byte-identical rerun tests	`cargo test`
Look-ahead	Causality and availability tests	model/strategy tests
Local data loss	Export/backup and restore test	22 memory tests, including round-trips
Linux packaging failure	Installer smoke test	Ubuntu 24.04 container install + launch
Scope creep into cloud	Plan scope fence and ADR	plan/ADR
Strategy IP exposure	No network in engines; local-only data	offline AppImage launch

Non-test control: the human review gate and the scope-fence ADR.

1.3 One behaviour

Expected: the same fixture and config produce the same evidence bundle bytes on two runs.

Observed: repeated `report model` bundles and repeated `run check` output are byte-identical; the full Rust suite passes 1,476 tests.

Limit: this proves the fixture and the local machine, not every platform or dataset.

1.4 Review and AI use

- Review: the plan and the PR are reviewed at `In Review`.
- AI use: recorded in the AI transparency log; the human verifies the result.
- Residual risk: the browser E2E uses a fixture backend; the native smoke does not drive the UI. The clean install is containerized, not bare-metal.

1.5 Decision

Keep the quality gates. The Phase 3 evidence is recorded in `docs/reports/2026-11-11/implementation-verificat` add a native WebDriver flow only if release risk requires it.